

Module 1

1. Explain RDBMS features. [3 marks]

A Relational Database Management System (RDBMS) stores data in a structured format using rows and columns. Its key features include:

- **Table Structure:** Data is organized into logical tables (relations) rather than flat files.
- **Data Integrity:** Enforces rules (like Primary Keys and Foreign Keys) to ensure data accuracy and consistency.
- **ACID Compliance:** Guarantees that database transactions are processed reliably (Atomicity, Consistency, Isolation, Durability).
- **Multi-User Access:** Allows multiple users to query and update the database simultaneously without data conflict.

2. Explain 2 Tier database architecture. [3 marks]

In a **2-Tier Architecture**, the system is divided into two parts: the client and the server.

- **Client Tier:** This is the user interface or application (like a desktop application) where the user inputs data.
- **Server Tier:** This is the database machine that processes queries and manages the data.
- **Direct Communication:** The client application connects directly to the database server without any middleware or application server in between, making it fast for small-scale applications but harder to scale for thousands of users.

3. What is the purpose of E-R Diagram? Explain One-to-Many relationship with suitable example. [5 marks]

An **Entity-Relationship (E-R) Diagram** is a visual blueprint of a database. Its purpose is to map out the logical structure of a database by showing **Entities** (objects/tables), **Attributes** (properties/columns), and the **Relationships** between those entities.

One-to-Many Relationship: A one-to-many relationship occurs when a single record in a parent table is associated with multiple records in a child table.

- **Example:** A **Departments and Employees** relationship.
- One parent departments table (containing dept_id and dept_name) can have many employees.
- The child employees table uses a Foreign Key (dept_id) to link back to the department, allowing multiple employees to belong to a single department.

4. What are the characteristics of Relational Model? [3 marks]

- **Data Representation:** All data is represented as two-dimensional tables called **Relations**.
- **Tuples and Attributes:** Rows are called **Tuples** (representing a single record), and columns are called **Attributes** (representing a specific field).
- **Uniqueness:** No two duplicate rows are allowed in a valid relation; every tuple must be uniquely identifiable using a primary key.
- **Order Insensitivity:** The order of the rows and columns does not matter to the database.

5. Explain Schema Mapping. [5 marks]

Schema Mapping is the process of transforming a higher-level, abstract database design into a lower-level, implementable database structure.

- **Conceptual to Logical:** Converting an E-R Diagram (entities and relationships) into a Relational Schema (tables, primary keys, and foreign keys).
- For instance, an entity like "Student" becomes a table, its properties become columns, and a many-to-many relationship might map into a new junction table.
- **Logical to Physical:** Mapping the relational tables to physical file storage structures on the actual disk. Schema mapping ensures that the business requirements visually designed in the conceptual phase are correctly translated into functioning SQL code.

6. What is Primary Key, Foreign Key and Candidate Key. [3 marks]

- **Primary Key:** A column (or set of columns) that uniquely identifies every row in a table. It cannot contain NULL values. For example, stud_id in a student table.
- **Foreign Key:** A column used to link two tables together by referencing the Primary Key of a parent table.
- **Candidate Key:** A column, or set of columns, that has the potential to be a primary key because it uniquely identifies rows (e.g., an Employee ID and an SSN are both candidate keys, though only one will be chosen as the primary key).

7. What is Functional Dependency and explain its types. [3 marks]

Functional Dependency (FD) is a relationship between two attributes in a table, typically written as $X \rightarrow Y$. This means that the value of attribute X uniquely determines the value of attribute Y.

- **Trivial Dependency:** Occurs when Y is a subset of X (e.g., {Student_ID, Name} $\rightarrow$ Name).
- **Non-Trivial Dependency:** Occurs when Y is not a subset of X (e.g., Student_ID $\rightarrow$ Name).
- **Partial Dependency:** Occurs when an attribute depends on only a part of a composite primary key.
- **Transitive Dependency:** Occurs when a non-key attribute depends on another non-key attribute ($X \rightarrow Y$, and $Y \rightarrow Z$, so $X \rightarrow Z$).

8. Explain Normalization with appropriate Example. [5 marks]

Normalization is the process of organizing database tables to minimize data redundancy and eliminate anomalies (insert, update, delete anomalies).

- **1NF (First Normal Form):** Ensures all columns contain atomic (indivisible) values and there are no repeating groups.
- **2NF (Second Normal Form):** Must be in 1NF, and all non-key attributes must be fully functionally dependent on the entire primary key (removes partial dependency).
- **3NF (Third Normal Form):** Must be in 2NF, and non-key attributes must not depend on other non-key attributes (removes transitive dependency).
- **Example:** Imagine an unnormalized Employee table containing: Emp_ID, Emp_Name, Dept_Name, Dept_Location. If 50 employees work in the "IT" department, the location "New York" is typed 50 times. To normalize this, we split it into two tables: an Employee table (Emp_ID, Emp_Name, Dept_ID) and a Department table (Dept_ID, Dept_Name, Dept_Location). This removes redundancy.

Module 2

1. Explain Insert, Update, Delete Command with suitable examples. [3 marks]

These are standard Data Manipulation Language (DML) commands used to modify table data.

- **INSERT:** Adds new rows into a table. *Example:* INSERT INTO dept (deptno, dname, location) values ('10', 'computer', 'NewYork');
- **UPDATE:** Modifies existing data within a table. *Example:* UPDATE emp SET Salary = 80000 WHERE eno = 213;
- **DELETE:** Removes existing rows from a table based on a condition. *Example:* DELETE FROM emp WHERE eno = 213;

2. What is Joins? Explain types of Joins. [5 marks]

A **JOIN** is an SQL operation used to combine rows from two or more tables based on a related column between them.

- **Inner Join:** Combines rows from both tables as long as there is an exact match between the shared columns.
- **Left (Outer) Join:** Returns all rows from the left table, and the matched rows from the right table. If there is no match, the result is NULL on the right side.
- **Right (Outer) Join:** Returns all rows from the right table, and the matched rows from the left table.
- **Full (Outer) Join:** Returns all rows when there is a match in either the left or right table.

3. Explain Subquery. [5 marks]

A **Subquery** is simply a query nested inside another query.

- The database runs the "inner" query first, then uses that result to filter the "outer" main query.
- **Example Usage:** If you want to find employees located in 'NewYork' without using an INNER JOIN, you can use a subquery.

SQL Example:

SQL

```
SELECT ename FROM emp
```

```
WHERE deptno IN (
```

```
SELECT deptno FROM dept WHERE location = 'NewYork'
```

```
);
```

The inner query finds the department number for New York, and the outer query selects the employees working in that department.

4. Explain VIEW. [5 marks]

A **VIEW** is a virtual table based on the result-set of an SQL query.

- **Characteristics:** It does not store data physically; instead, it saves the SQL query itself. When you select from a view, the database runs the underlying query to fetch the data dynamically.
- **Purpose:** Views are used for security (restricting user access to specific columns/rows rather than the whole table), simplifying complex queries, and presenting aggregate data cleanly.
- **Syntax Example:** CREATE VIEW HighEarnings AS SELECT Ename, Salary FROM EMP WHERE Salary > 60000;

5. What are the advantages of PLSQL Function. How to create a Function in PLSQL? Give Syntax. [3 marks]

A **PL/SQL Function** is a named block of code that performs an action and **must** return a single value.

- **Advantages:** Promotes code reusability, improves performance by running directly on the database server, and allows complex business logic to be modularized.

Syntax:

SQL

```
CREATE OR REPLACE FUNCTION function_name (parameter_name datatype)
```

```
RETURN return_datatype IS
```

```
-- variable declarations
```

```
BEGIN
```

```
-- execution logic
```

```
RETURN value;
```

```
END;
```

6. Explain Trigger with suitable examples. [5 marks]

A **Trigger** is a stored procedure that the database automatically executes (or "fires") when a specific event occurs on a table or view.

- **Events:** Triggers are commonly attached to INSERT, UPDATE, or DELETE statements.
- **Purpose:** They are used to enforce strict business rules, perform automatic data validations, and maintain audit trails (like tracking who changed a record and when).
- **Example Use Case:** A trigger that automatically updates an inventory table, reducing the stock quantity whenever a new record is inserted into a sales table.

7. Create a Trigger by keeping old data in one table before Updating. [3 marks] To keep a history of changed records, we use a BEFORE UPDATE trigger that inserts the :OLD values into an audit table.

SQL

```
CREATE OR REPLACE TRIGGER trg_audit_emp
```

```
BEFORE UPDATE ON emp
```

```
FOR EACH ROW
```

```
BEGIN
```

```
INSERT INTO emp_audit_table (eno, old_salary, change_date)
```

```
VALUES (:OLD.eno, :OLD.salary, SYSDATE);
```

```
END;
```

8. Explain ACID Properties. [3 marks]

ACID properties ensure that database transactions are processed reliably, even in the event of errors or power failures.

- **Atomicity:** The "all or nothing" rule. A transaction completely succeeds, or it completely fails.
- **Consistency:** Data must be valid according to all defined rules (like constraints) before and after the transaction.
- **Isolation:** Concurrent transactions operate independently without interfering with one another.
- **Durability:** Once a transaction is committed, it remains saved permanently, even if the system crashes.

9. Explain Commit, Rollback and Savepoint. [3 marks]

These are Transaction Control Language (TCL) commands used to manage database transactions.

- **COMMIT:** Permanently saves all changes made during the current transaction to the database.
- **ROLLBACK:** Undoes all changes made during the current transaction, reverting the database to its previous state.
- **SAVEPOINT:** Sets a specific marker within a transaction. It allows you to roll back a portion of the transaction (to the savepoint) rather than rolling back the entire transaction.